

CS 185/285 Deep Reinforcement Learning

Assignment 2: Policy Gradients

Experimental Report

This report summarizes the required experiments for CartPole-v0, HalfCheetah-v4, LunarLander-v2, and InvertedPendulum-v4. All learning curves use environment steps on the x-axis, as required by the assignment.

Section	Environment	Main topic
3.2	CartPole-v0	Reward-to-go, advantage normalization, batch size
4.2	HalfCheetah-v4	Neural-network baseline
5	LunarLander-v2	Generalized Advantage Estimation (GAE)
6	InvertedPendulum-v4	Hyperparameter tuning and sample efficiency

3.2 Vanilla Policy Gradients - CartPole-v0

Eight policy-gradient runs were compared using trajectory-centric returns versus reward-to-go (RTG), with and without advantage normalization, at batch sizes 1000 and 4000.

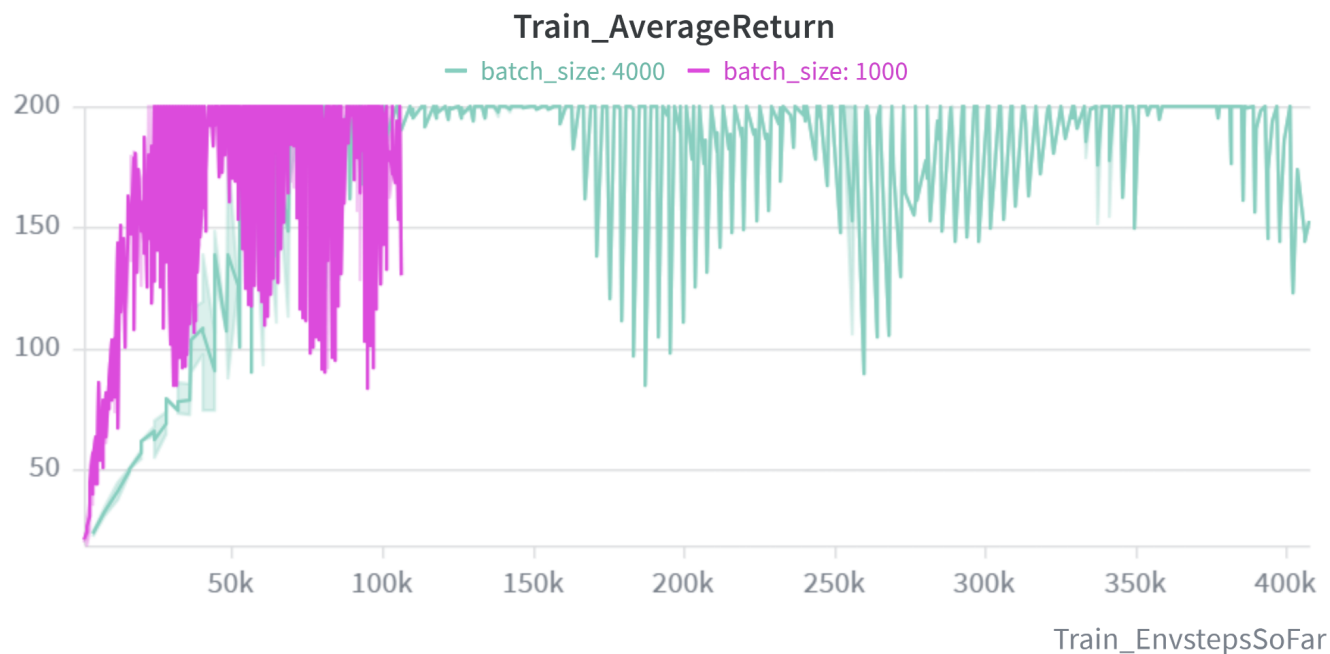


Figure 1. Overall CartPole batch-size comparison (batch size 1000 vs. 4000).

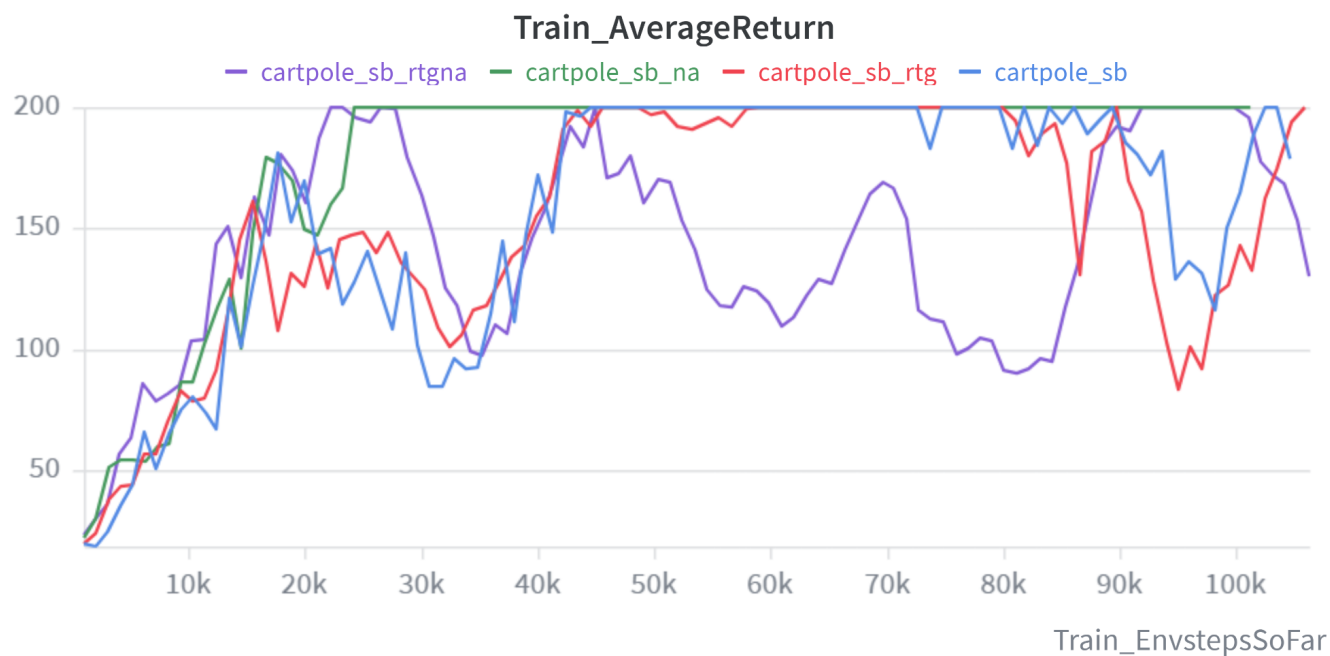


Figure 2. Small-batch CartPole experiments (batch size = 1000).

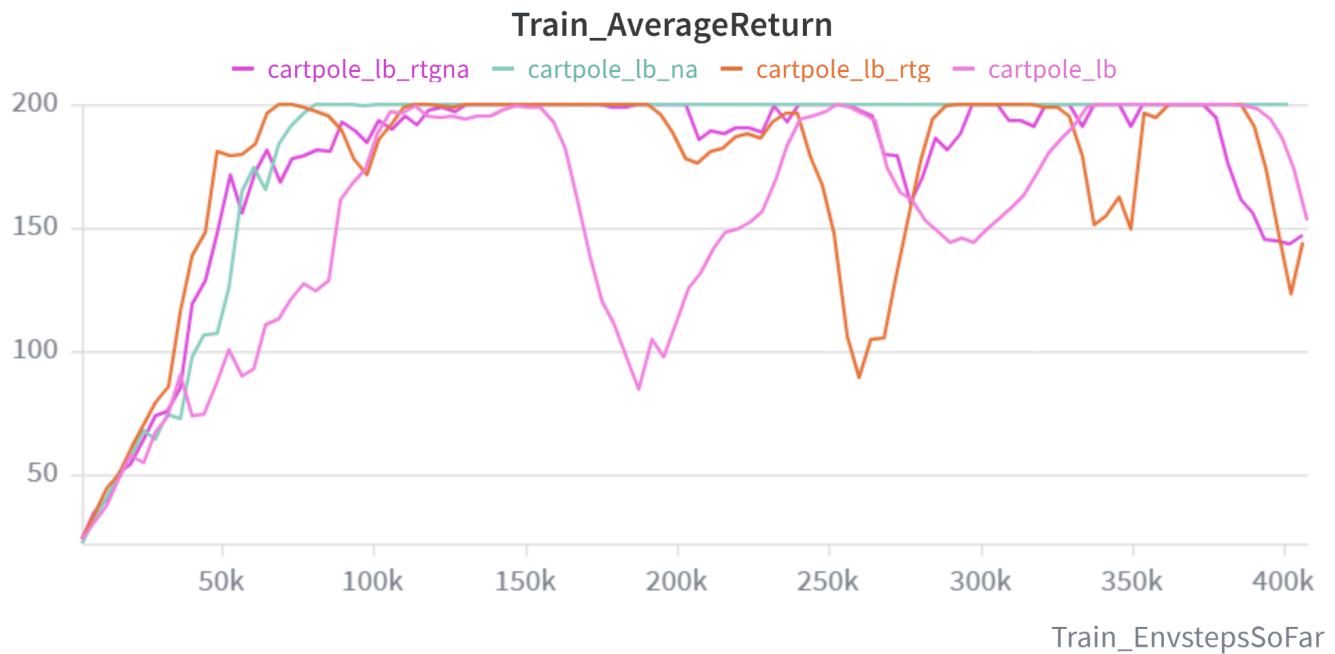


Figure 3. Large-batch CartPole experiments (batch size = 4000).

Questions and Answers

Q3-1. Which value estimator has better performance without advantage normalization?

For the large batch size, reward-to-go (RTG) performed better, converging to the maximum return of 200 approximately 30k steps earlier than the trajectory-centric estimator. For the small batch size, there was no significant difference between the two estimators.

Q3-2. Why is one estimator generally preferred over the other?

Reward-to-go is generally preferred because it reduces the variance of the policy-gradient estimator. It only considers rewards obtained after the current action, since rewards received before that action cannot be influenced by it.

Q3-3. Did advantage normalization help?

Yes. Advantage normalization made training more stable by normalizing the scale of the advantage estimates, resulting in more consistent policy updates.

Q3-4. Did the batch size make an impact?

Yes. The small batch size produced much larger fluctuations during training, while the large batch size produced more stable learning. However, the small batch converged faster, at approximately 50k environment steps, whereas the large batch generally converged at around 100k steps.

Exact Command Line Configurations

```
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 1000 --exp_name cartpole
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 1000 -rtg --exp_name cartpole_rtg
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 1000 -na --exp_name cartpole_na
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 1000 -rtg -na --exp_name cartpole_rtg_na

uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 4000 --exp_name cartpole_lb
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 4000 -rtg --exp_name cartpole_lb_rtg
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 4000 -na --exp_name cartpole_lb_na
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 4000 -rtg -na --exp_name cartpole_lb_rtg_na
```

4.2 Using a Neural Network Baseline - HalfCheetah-v4

The baseline experiment evaluates whether a learned state-value function improves the policy-gradient advantage estimate. An additional run reduces the number of baseline gradient steps from 5 to 1 to test how insufficient baseline fitting affects learning.

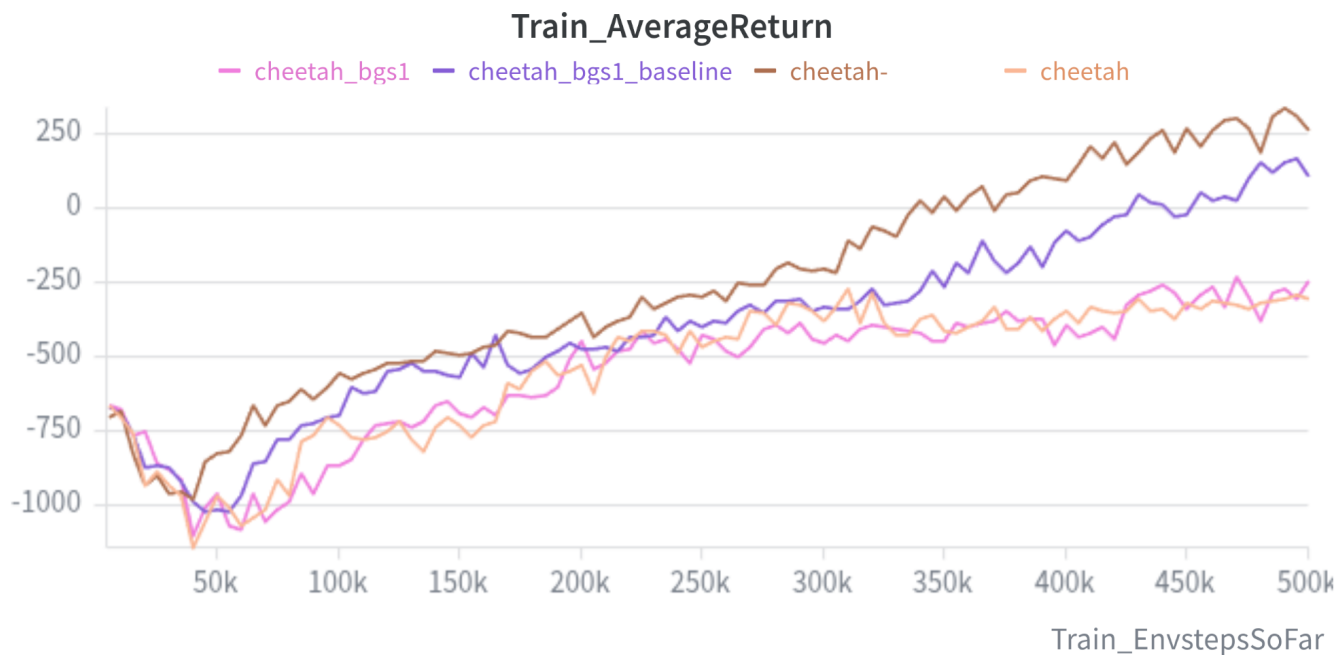


Figure 4. HalfCheetah training-return comparison across the tested baseline configurations.

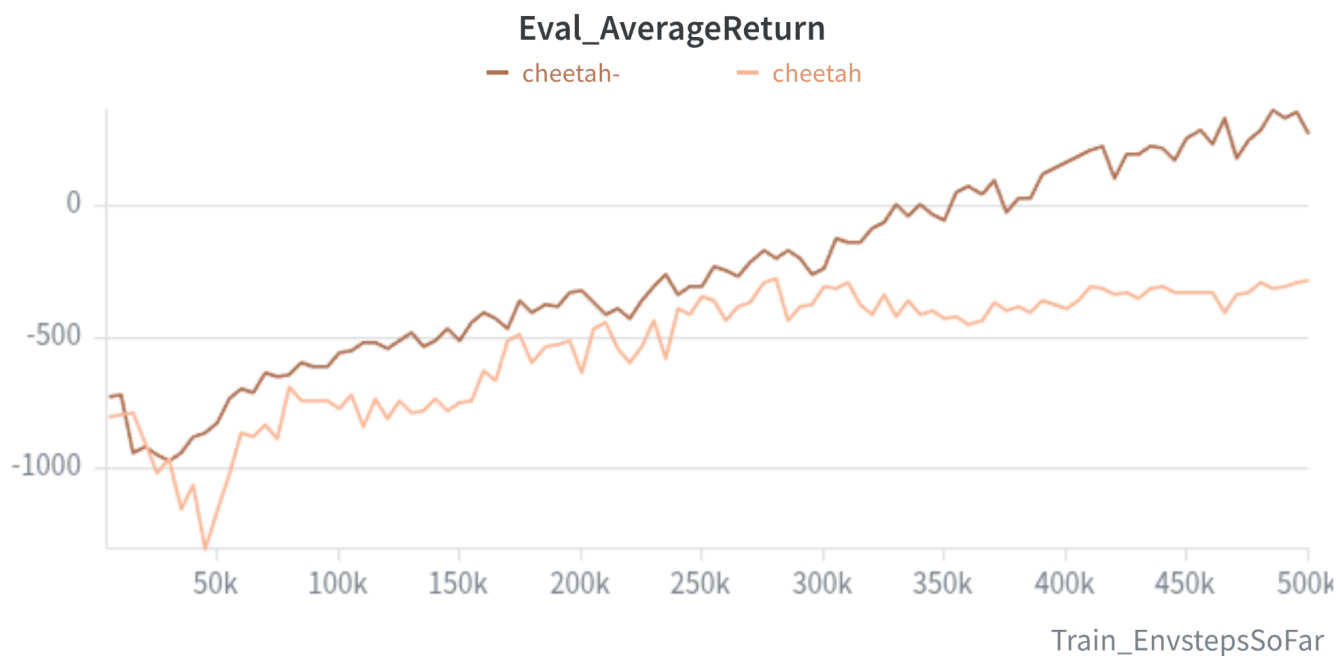


Figure 5. Baseline loss learning curve.

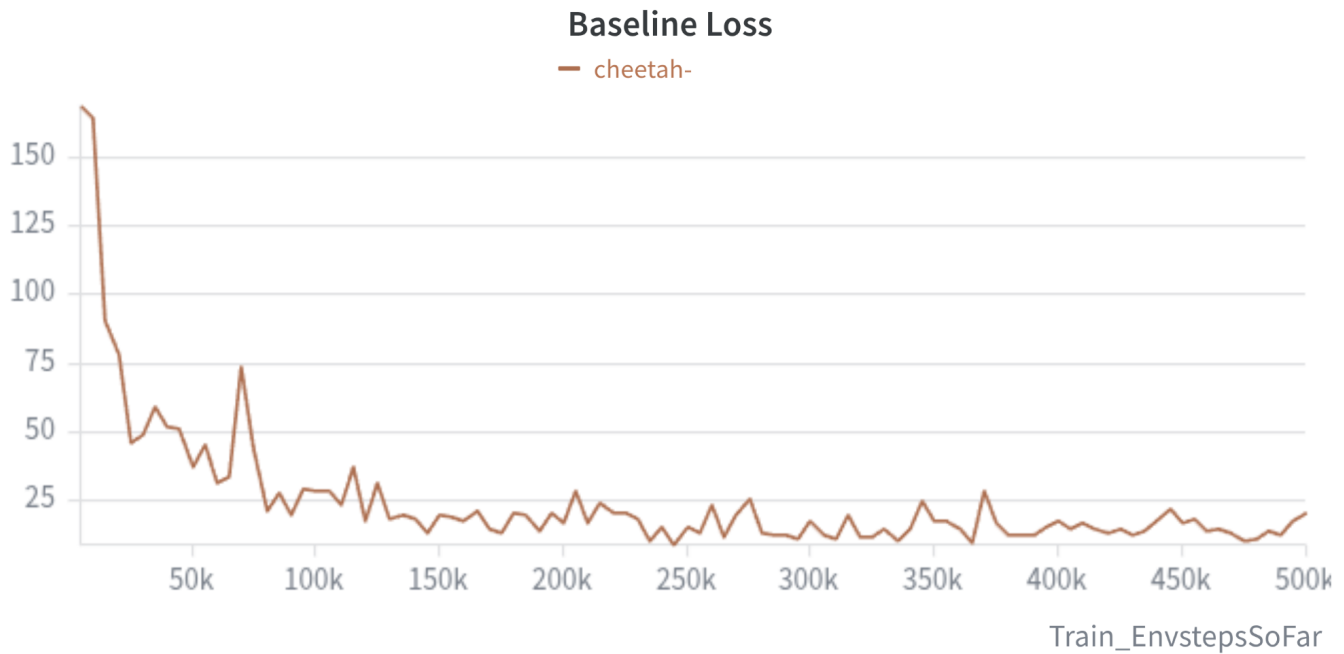


Figure 6. HalfCheetah evaluation-return comparison.

Effect of Decreasing Baseline Gradient Steps

Decreasing the number of baseline gradient steps (-bgs) from 5 to 1 significantly degraded policy performance. The original configuration with bgs=5 reached an evaluation return of approximately 300, whereas the configuration with bgs=1 remained around -300.

The baseline learning curve also showed poorer convergence with fewer gradient steps, indicating that the baseline was not trained sufficiently to accurately estimate the value function. Consequently, the advantage estimates became less accurate, leading to less effective policy updates and substantially worse policy performance.

Exact Command Line Configurations

```
# No baseline
uv run src/scripts/run.py --env_name HalfCheetah-v4 -n 100 -b 5000 -eb 3000 -rtg \
--discount 0.95 -lr 0.01 --exp_name cheetah

# Baseline, original bgs=5
uv run src/scripts/run.py --env_name HalfCheetah-v4 -n 100 -b 5000 -eb 3000 -rtg \
--discount 0.95 -lr 0.01 --use_baseline -blr 0.01 -bgs 5 \
--exp_name cheetah_baseline

# Reduced baseline gradient steps
uv run src/scripts/run.py --env_name HalfCheetah-v4 -n 100 -b 5000 -eb 3000 -rtg \
--discount 0.95 -lr 0.01 --use_baseline -blr 0.01 -bgs 1 \
--exp_name cheetah_baseline_bgs1

# Additional bgs=1 run without baseline flag
uv run src/scripts/run.py --env_name HalfCheetah-v4 -n 100 -b 5000 -eb 3000 -rtg \
--discount 0.95 -lr 0.01 -blr 0.01 -bgs 1 --exp_name cheetah_bgs1
```

5. Generalized Advantage Estimation - LunarLander-v2

GAE was evaluated with lambda in $\{0, 0.95, 0.98, 0.99, 1\}$. The experiment probes the bias-variance tradeoff of the advantage estimator while keeping the remaining hyperparameters fixed.

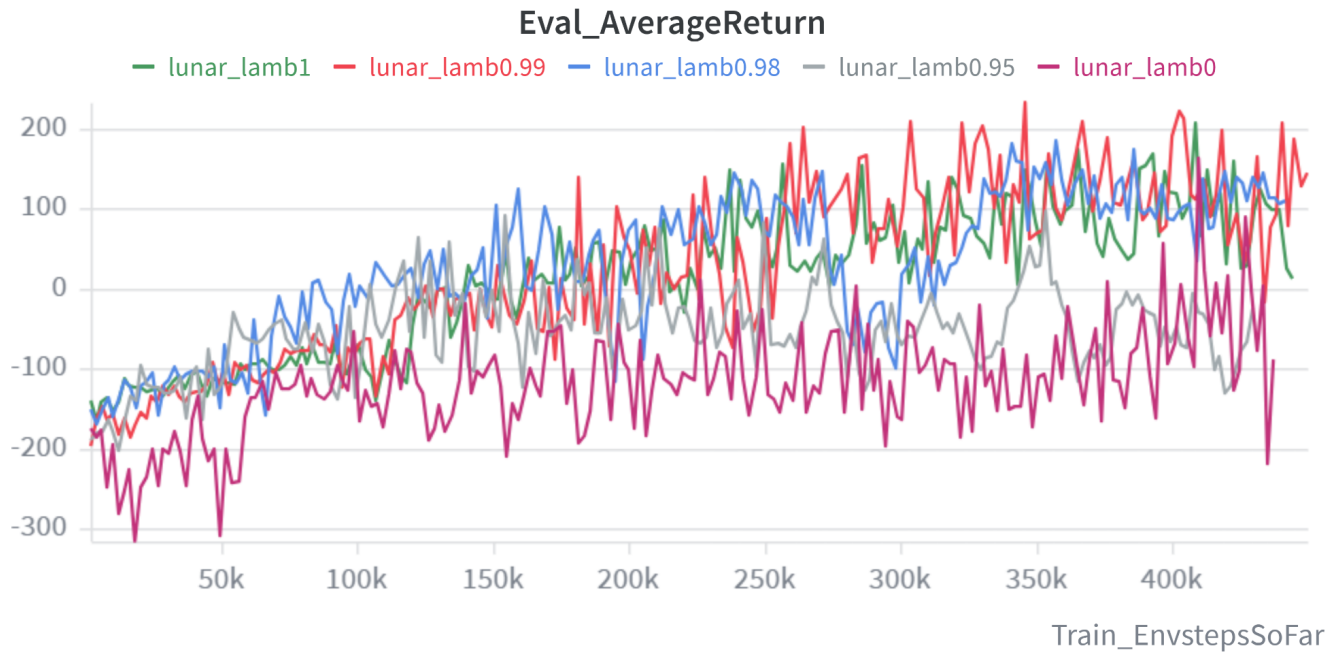


Figure 7. LunarLander evaluation average return for the five GAE lambda values.

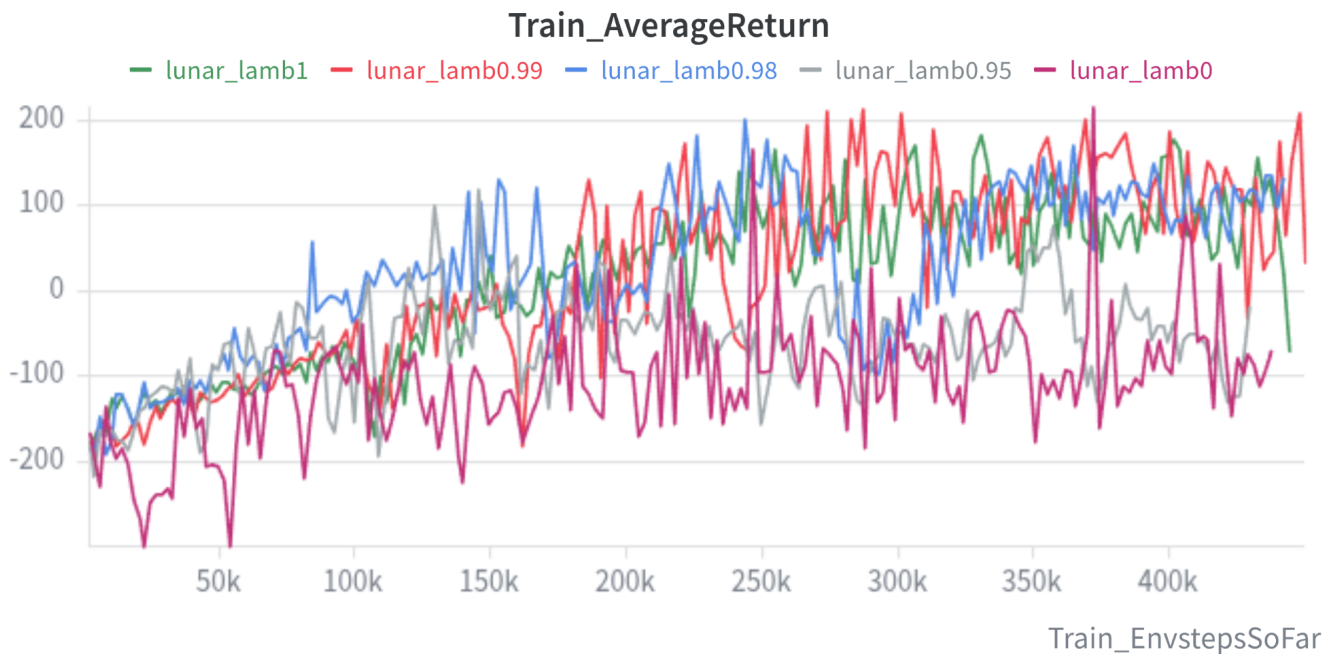


Figure 8. LunarLander training average return for the five GAE lambda values.

Effect of lambda on Task Performance

Larger lambda values generally led to better task performance. $\lambda = 0$ performed significantly worse and failed to learn a good policy, while values close to 1 achieved substantially higher returns. In particular, $\lambda = 0.98-0.99$ showed relatively strong performance.

Interpretation of lambda

$\lambda = 0$ corresponds to the one-step TD advantage estimate, which has higher bias but lower variance, while $\lambda = 1$ corresponds to the Monte Carlo-style advantage estimate using the full trajectory return, which has lower bias but higher variance. In LunarLander-v2, $\lambda = 0$ performed poorly, whereas lambda values close to 1 achieved substantially better performance.

Exact Command Line Configurations

```
uv run src/scripts/run.py --env_name LunarLander-v2 --ep_len 1000 --discount 0.99 \
-n 200 -b 2000 -eb 2000 -l 3 -s 128 -lr 0.001 --use_reward_to_go --use_baseline \
--gae_lambda 0 --exp_name lunar_lander_lambda0
```

```
uv run src/scripts/run.py --env_name LunarLander-v2 --ep_len 1000 --discount 0.99 \
-n 200 -b 2000 -eb 2000 -l 3 -s 128 -lr 0.001 --use_reward_to_go --use_baseline \
--gae_lambda 0.95 --exp_name lunar_lander_lambda0.95
```

```
uv run src/scripts/run.py --env_name LunarLander-v2 --ep_len 1000 --discount 0.99 \
-n 200 -b 2000 -eb 2000 -l 3 -s 128 -lr 0.001 --use_reward_to_go --use_baseline \
--gae_lambda 0.98 --exp_name lunar_lander_lambda0.98
```

```
uv run src/scripts/run.py --env_name LunarLander-v2 --ep_len 1000 --discount 0.99 \
-n 200 -b 2000 -eb 2000 -l 3 -s 128 -lr 0.001 --use_reward_to_go --use_baseline \
--gae_lambda 0.99 --exp_name lunar_lander_lambda0.99
```

```
uv run src/scripts/run.py --env_name LunarLander-v2 --ep_len 1000 --discount 0.99 \
-n 200 -b 2000 -eb 2000 -l 3 -s 128 -lr 0.001 --use_reward_to_go --use_baseline \
--gae_lambda 1.0 --exp_name lunar_lander_lambda1.0
```

6. Hyperparameters and Sample Efficiency - InvertedPendulum-v4

The goal was to reach the maximum evaluation return of 1000 within 100K environment steps. Batch size, learning rate, reward-to-go, and advantage normalization were tuned relative to the default configuration.

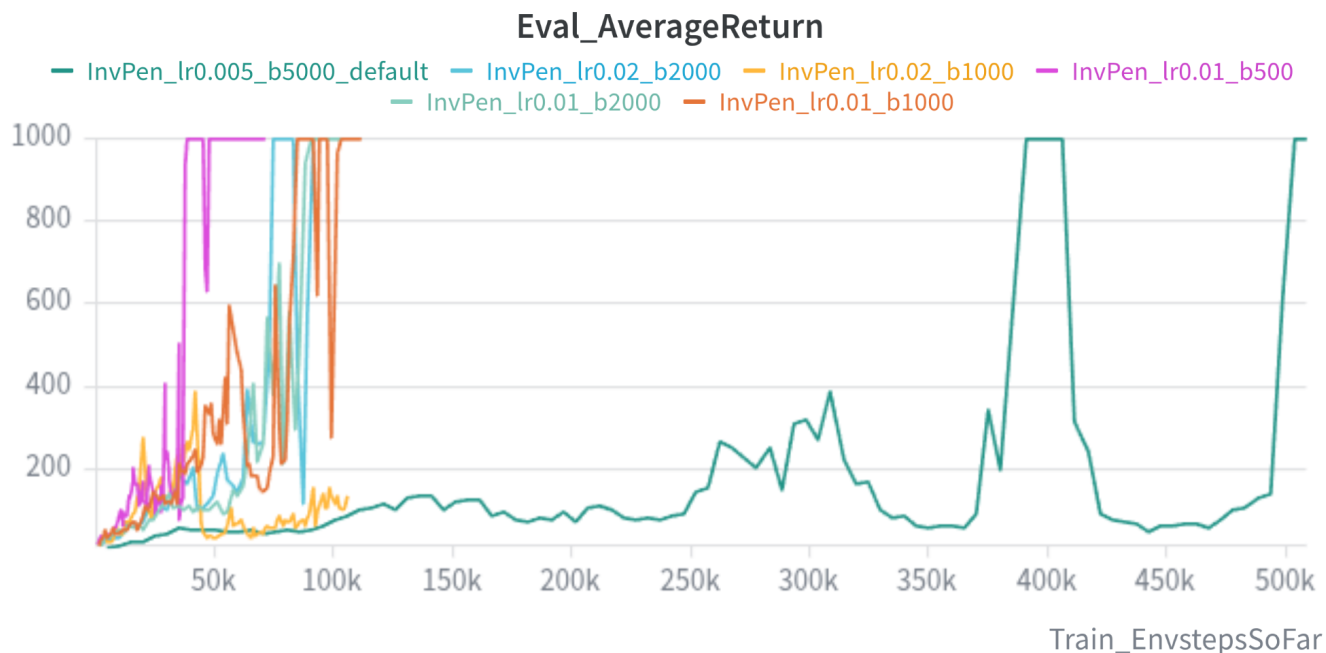


Figure 9. InvertedPendulum tuning runs including the default configuration.

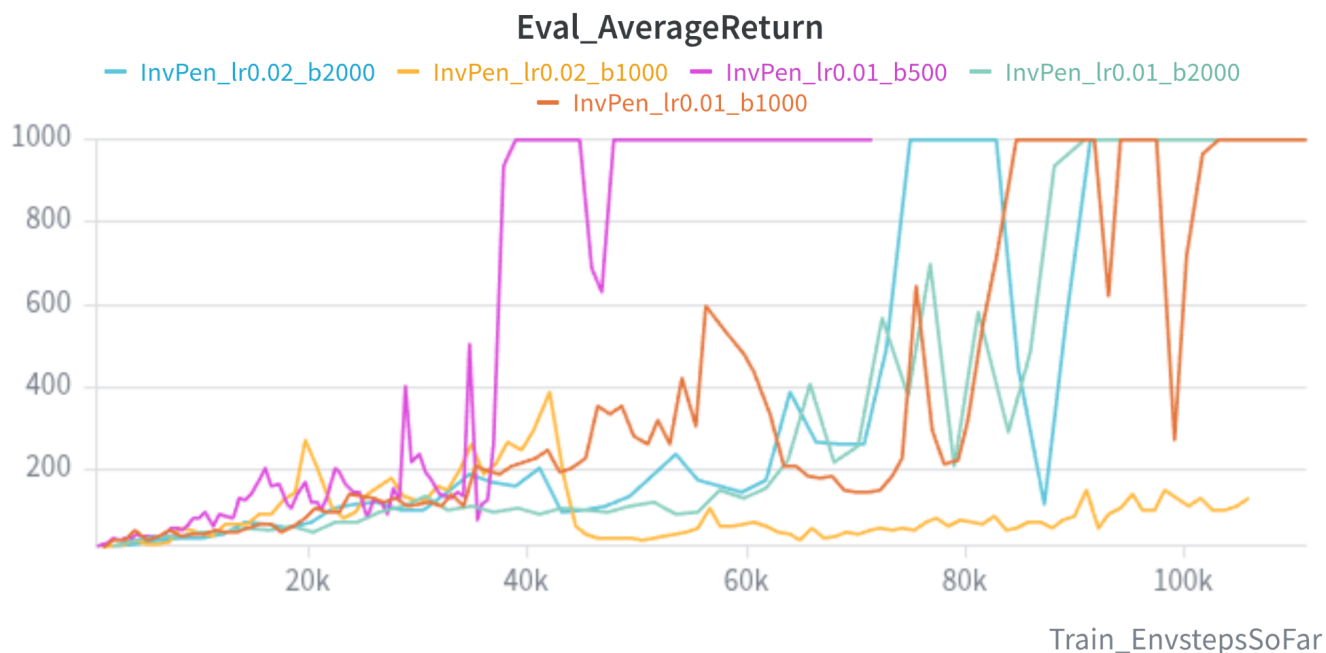


Figure 10. Detailed comparison of tuned InvertedPendulum runs within roughly the first 110K environment steps.

Best Hyperparameters

Hyperparameter	Best value
batch_size	500
learning_rate	0.01
use_reward_to_go	True
normalize_advantages	True

This configuration reached the maximum average return of 1000 after approximately 39K environment steps.

Brief Discussion

The best configuration (batch_size=500, learning_rate=0.01) converged substantially faster than the default configuration (batch_size=5000, learning_rate=0.005), which required approximately 390-400K environment steps to reach an average return of 1000.

Among the hyperparameters tested, batch size had the largest effect on sample efficiency. Reducing the batch size allowed more frequent policy updates for the same number of environment interactions, leading to much faster convergence. Increasing the learning rate from 0.01 to 0.02 did not consistently improve performance.

Exact Best-Run Command

```
uv run src/scripts/run.py --env_name InvertedPendulum-v4 -n 100 -b 500 -eb 1000 \
-rtg -na --learning_rate 0.01 --exp_name pendulum_b500
```

Default Command for Comparison

```
uv run src/scripts/run.py --env_name InvertedPendulum-v4 -n 100 -b 5000 -eb 1000 \
--exp_name pendulum
```